



University
of Glasgow

WORLD
CHANGING
GLASGOW

Embedded System Testing, and Debugging

Dr Sajjad Hussain

Associate Professor

UESTC 2004: Embedded Processors

Slides by: Dr Yao Sun





- Dr. Sajjad Hussain, Associate Professor, Glasgow College
- sajjad.hussain.2@glasgow.ac.uk
- Research area: Wireless Communications (5G/6G Physical layer design, Phased Arrays)



Aim

- Explain Embedded System Software Development, Testing, and Debugging

Learning Objectives

- Understanding the different types of testing and different testing techniques
- Identify the debugging tools used for embedded systems

Assumed Previous Knowledge

- Understanding of the instruction set of ARM microcontroller,
- Understanding of assembly language basics
- C programming.

- Complex systems need testing to ensure that they work as they are intended.
- But bugs can be subtle, particularly in embedded systems, where **specialised hardware and real-time responsiveness** make programming more challenging.
- Fortunately, there are many available techniques for software testing that can help us generate a comprehensive set of tests to ensure that our system works properly



Types of Testing

- **Unit testing:** Testing **individual software units** (functions, modules) in isolation to verify their correctness and functionality.
- **Integration testing:** Testing the interaction and integration **between different software modules or components** to ensure they work together as intended.
- **System testing:** Testing the complete **embedded system as a whole** to validate its overall behaviour and functionality.
- **Acceptance testing:** Testing performed to ensure the embedded system **meets customer or user requirements** and satisfies their expectations.
- **Regression testing:** **Repeating previously conducted tests** to ensure that changes or fixes in the software did not introduce new defects or issues.

White Box Testing

- White box testing is also called structural testing.
- White Box Testing is software testing technique in which **internal structure, design and coding** of software are tested to verify flow of input-output and to improve design, usability and security.

The primary objective of white box testing is to verify the quality of the application code.

Since the **system's internal structure is known**, developers can pinpoint where errors come from.



A Car mechanic should know the internal structure of the car engine to repair it!

- Example: cruise control white box test
 - Test operation at every point in control loop lookup table
 - Tests that exercise both paths of every conditional branch statement



Aspects of the program that can be verified using **White-Box** testing:

- Broken or poorly structured paths in the coding processes
- Internal security holes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object, and function on an individual basis



Advantages of White Box Testing

- Code optimization by **finding hidden errors**.
- White box tests cases can be easily **automated**.
- Testing is more thorough as all code paths are usually covered.
- Testing can **start early** in system development life cycle even if GUI is not available.

Disadvantages of White Box Testing

- White box testing can be **quite complex and expensive**.
- If the white box testing by developers is **not detailed** can **lead to production errors**.
- White box testing requires **professional resources**, with a **detailed understanding** of programming and implementation.
- White-box testing **is time-consuming**, bigger programming applications take the time to test fully. 8

Black Box Testing

- Black box testing is also called **functional testing** or **data-driven testing**.
- The object of this approach is to **check program functionality based on input-output**.
- Since black box testers do not know how the program works, their concerns reflect those of regular users. **This testing method is based on trial and error.**
- In the black-box testing, major **testing will be around possible inputs and expected outputs.**



A simple example of black-box testing is a TV

Example: cruise control black box test

- Test operation at various speeds
- Test operation at various underspeed/overspeed amounts
- BUT, no knowledge of whether lookup table or control equation is used



Aspects of the program that can be verified using **Black-Box** testing:

- Functional Requirements
- Input Validation
- Usability and User Interface
- Error Handling and Exception Handling
- Performance and Scalability
- Security and Vulnerability Assessment
- Compatibility and Interoperability



Advantages of Black Box Testing

- Well suited and **efficient** for large code segments.
- **Code Access** not required.
- Clearly separates **user's perspective** from the **developer's perspective** through visibly defined roles.
- Large numbers of **moderately skilled testers** can test the application with no knowledge of implementation, programming language or operating systems.

Disadvantages of Black Box Testing

- **Limited Coverage** since only a selected number of test scenarios are actually performed.
- **Inefficient testing** because the tester only has limited knowledge about an application.
- **Blind Coverage**, since the tester cannot target specific code segments or error prone areas.
- The **test cases** are difficult to design.

Grey Box Testing

- Grey-box testing is a testing technique that **combines** elements of both black-box testing and white-box testing.
- It involves **having partial knowledge of the internal structure** or implementation details of the program or system being tested while **still maintaining a focus on the external behaviour**.



A simple example of grey-box testing is a web application:

- Testers may have access to the application's **source code**, allowing them to understand the underlying **logic and database interactions**.
- With this knowledge, they can **design test cases** to verify specific paths, conditions, or edge cases within the application.
- For instance, **they could test a login feature** by examining the code responsible for user authentication and designing test cases based on different user roles or specific database states.

Advantages of Grey Box Testing

- **Increased Test Coverage:** allows testers to design test cases based on both the external behaviors and the internal code paths, providing broader coverage than black-box testing alone.
- **Efficient Bug Localization:** With access to internal details, grey-box testing can help pinpoint the root cause of a bug or issue more accurately
- **Optimal Use of Resources:** Grey-box testing allows for a balance between the benefits of white-box testing (e.g., thoroughness) and black-box testing (e.g., independence).

Disadvantages of Grey Box Testing

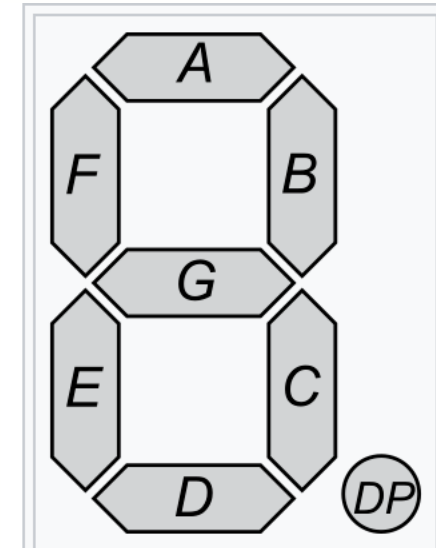
- **Limited Visibility:** Does not provide complete visibility into the internal workings of the system, some potential issues may remain undetected.
- **Dependencies on Internal Knowledge:** Grey-box testing relies on access to internal information, this may pose challenges if such information is not readily available or accessible.

Objective:

Design and implement an embedded system using the Nucleo L432KC board and Mbed that allows users to increment and decrement a counter displayed on a 7-segment display using two push buttons.

Decimal Numbers

0 1 2 3 4 5 6 7 8 9



Segment names of a seven-segment display with an eighth Decimal Point segment.

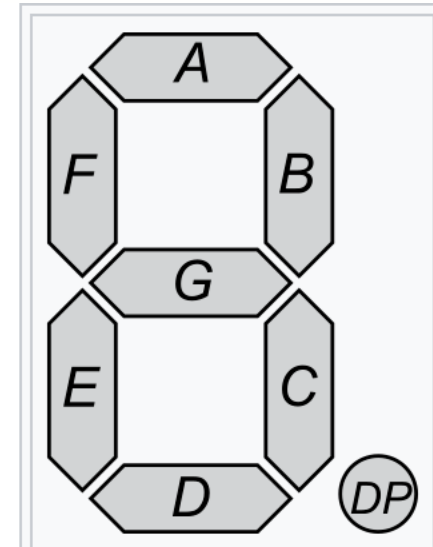
White box testing?

Statement Coverage:

- Design test cases to ensure that **each line of code** responsible for controlling segments of the 7-segment display is executed at least once during testing.
- Verify that the code correctly initializes the segments and updates them according to the input data.

Branch Coverage:

- Test different scenarios to ensure that all **conditional branches** in the code are exercised.
- For example, test the code paths for displaying different digits, handling special characters, or responding to error conditions.



Segment names of a seven-segment display with an eighth Decimal Point segment.

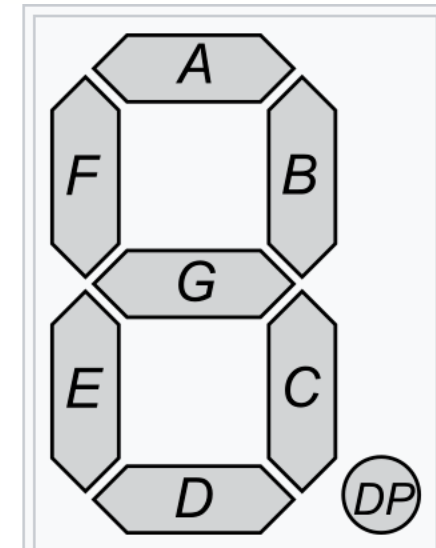
White box testing?

Path Coverage:

- Design test cases to cover all possible execution paths through the code.
- This involves considering different combinations of inputs and ensuring that the display behaves correctly in each case.

Code Inspection:

- Review the source code to identify potential errors or inefficiencies in the logic controlling the 7-segment display.
- Look for issues such as missing error handling, incorrect segment mappings, or inconsistent display behavior.



Segment names of a seven-segment display with an eighth Decimal Point segment.

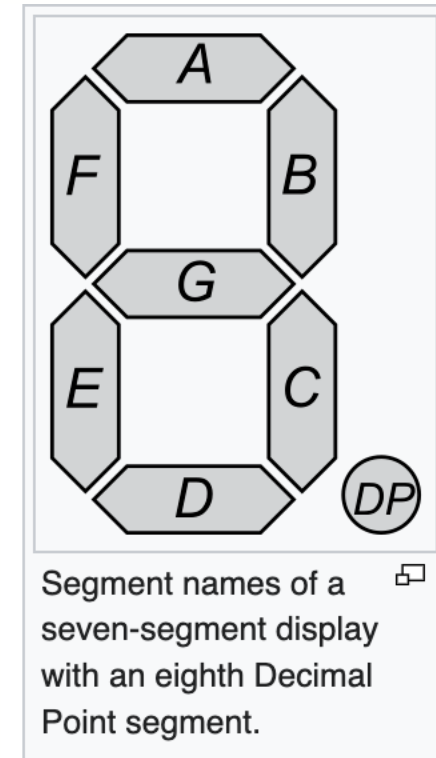
Black box testing?

Input Validation:

- Test the display with various types of input data, including valid digits, invalid characters, and out-of-range values.
- Verify that the display handles invalid inputs gracefully, such as displaying an error message or ignoring invalid characters

Boundary Testing:

- Test the display with inputs at the **extreme ends of the valid range**.
- For example, test the display with the lowest and highest possible values it can represent to ensure correct behavior at the boundaries



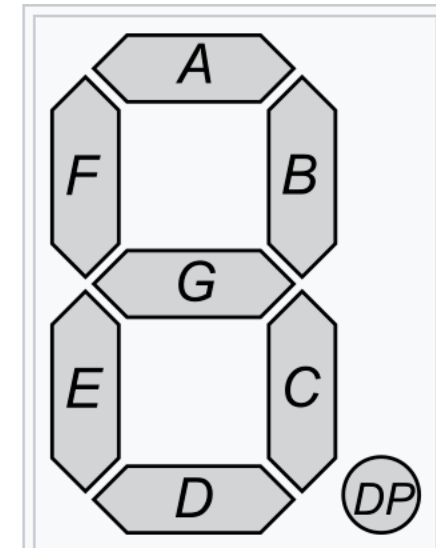
Black box testing?

Functional Testing:

- Verify that the display accurately represents the input data according to the specifications.
- Test scenarios such as displaying single digits, multiple digits, decimal points, and negative numbers to ensure correct functionality.

Integration Testing:

- Test the interaction between the 7-segment display and other components of the system, such as input devices or controllers.
- Ensure that data is correctly transferred to the display and that it responds appropriately to changes in input data.



Segment names of a seven-segment display with an eighth Decimal Point segment.

Static Testing

- Static testing is a testing technique that involves analysing software artifacts, such as firmware code, hardware specifications, device drivers, system requirements, and related documentation **without executing the actual program**.
- The purpose of static testing is to **identify defects, quality issues, and potential improvements early**.
- It is performed **by human testers or automated tools** without the need for executing the **whole system**.

In embedded systems:

- Static testing may involve reviewing firmware code for potential memory leaks, stack or heap overflows, real-time behavior violations, or adherence to coding standards specific to embedded systems.
- May also include **examining system requirements** and design documentation to ensure that the embedded system meets **safety standards, performance requirements, power constraints, and interoperability specifications**. (not just the product performance)



Advantages of Static Testing

- **Early Defect Identification:** Static testing helps identify potential defects, safety hazards, or performance issues early in the development process, reducing risks and costs associated with later-stage bug fixing or system failures.
- **Compliance and Safety:** Static testing enables verification of compliance with industry-specific standards
- **Optimization and Resource Management:** helps identify opportunities for optimizing code, reducing memory footprint, improving power efficiency, and ensuring efficient resource utilization in resource-constrained embedded systems.
- **Interoperability and Compatibility:** By reviewing hardware specifications, system interfaces, and communication protocols, static testing helps ensure interoperability and compatibility with other hardware or software components
- **Documentation Quality:** Static testing ensures the accuracy, completeness, and consistency of documentation systems.

Disadvantages of Static Testing

- **Limited Scope:** Static testing focuses on analysis and review, but it does not involve the actual execution of the software.
- **Human Error:** Static testing relies on human reviewers, making it susceptible to biases, oversights, and subjective judgments.
- **False Positives/Negatives:** Static analysis tools, if used, can generate false positives or false negatives.



Dynamic Testing

- Dynamic testing in embedded systems refers to the process of evaluating the behaviour and performance of an embedded **system by executing the software on the target hardware**.
- It involves **running tests** and **observing the system's response** to different inputs, stimuli, and operating conditions.
- Dynamic testing may involve executing test cases **on the actual hardware** or **on emulators/simulators** to assess the system's behaviour.
- This could include validating the **system's functionality**, measuring **response times**, analysing **real-time behaviour**, evaluating **power consumption**, or assessing the **system's performance** under various loads or stress conditions.

Advantages of Dynamic Testing

- Functional Validation
- Real-World Simulation
- Real-Time Behaviour Evaluation
- Performance Analysis
- Hardware-Software Integration

Disadvantages of Dynamic Testing

- Hardware and Environment Dependency
- Limited Coverage
- Complexity and Time Requirements
- Cost and Resource Intensiveness

Static Testing:

- **Analyzes** software artifacts without executing the program.
- Focuses on **early defect** identification, code quality, and adherence to standards.
- Performed during the **development phase**.
- Provides benefits such as efficient bug localization and compliance verification.

Dynamic Testing:

- **Executes** the software to evaluate its behavior and performance.
- **Verifies** functional behavior, real-time behavior, and performance.
- Conducted during the **testing phase**.
- Offers benefits such as **real-world simulation** and hardware-software integration verification.

Testing != Debugging

Two related goals: Are there any software defects? (if so, where)
We think the software is good to go –are we confident no defects remain?

Testing: Executing a program to **see if it performs as expected**

Debugging: **Given a symptom of software failure**, locate and correct the defect

Unpleasant truths: Testing tells you how buggy your software is,
NOT where **ALL** the bugs are

It's still useful know how to remove bugs





- ❑ Developing an embedded application requires hardware design, software coding and programming a system that is subject to real-world interactions.
- ❑ Hardware and software component must work together in an effective design.
- ❑ A debug tool can:
 1. help bring a prototype system up.
 2. help identify hardware and software problems both in the prototype and final application.
 3. assist in fine-tuning the system.

Factors in Choosing Debug Tools

Cost



Ease of use



Special features/capabilities





Debugging Tool Choices

1. Burn and Learn
2. Software Simulation
3. Oscilloscope
4. Logic Analyzer
5. In-Circuit Emulator (ICE)
6. In Circuit Debug

1. Burn and Learn

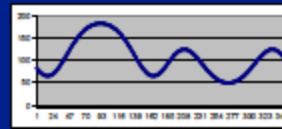
Debug cycle

1. Write code
 2. Program (burn) chip
 3. Test (learn)
 4. Erase and go to 1.
- Repeat



2. Software Simulation

- CPU and instruction set are simulated in software
- Most peripherals are simulated as well
- The simulator has simulated I/O:
 - Waveforms
 - Digital signals
 - Manual inputs
 - Register logs



3. Oscilloscope

- Verifies that signals are changing
- “Markers” can be inserted in code to verify that code execution reached certain points

Pros:

- Usually available in most design labs
- Easy to use
- Relatively inexpensive

Cons:

- Can monitor only a few signals on the pins of an embedded controller



4. Logic Analyzer

A logic analyzer is a hardware test instrument that is used to monitor digital signals. It can be a very good tool for debugging an embedded system with an external memory bus. A logic analyzer can log address, instruction and some data information as it executes.

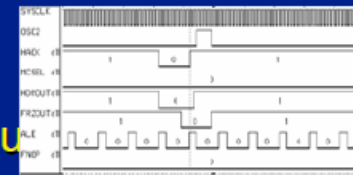
Suitable for debugging problems in interrupts

Pros:

- High speed
- Useful for designs with an external memory bus
- Can utilize a bus disassembler

Cons:

- Expensive
- No breakpoints
- Complex set up may be required
- Single chip microcontrollers allow monitoring of I/O only – not memory activity



5. Hardware Emulation

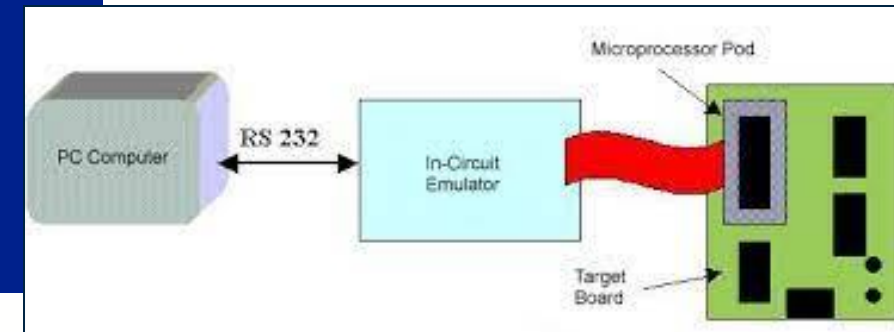
ICE: In-Circuit Emulation

- In-Circuit Emulator (ICE) takes the place of the target processor.
- It contains a copy of target processor, plus RAM, ROM, and its own embedded software.
- It allows you to examine the state of the processor while the program is running.
- It uses the remote debugger for human interface.
- It supports software and hardware breakpoints.
- It has real-time tracing.
- It stores the information about each processor cycle which is executed.
- It allows you to see in what order things happen.

5. Hardware Emulation

ICE: In-Circuit Emulation

- Specialized hardware required
- Replaces target microcontroller
- Pod program memory replaces microcontroller embedded memory
- Provides full visibility of memory, registers and CPU



6. In Circuit Debug (ICD)

Solves two hardware debug problems:

1. Cost of ICE
2. Increasing speed of embedded controllers

New benefits:

1. Can debug in-circuit
2. Can reprogram in-circuit





University
of Glasgow



WORLD
CHANGING
GLASGOW

Summary

- Understanding the testing types and techniques
- Identify testing and debugging tools



In the Next Lesson:

- Testing and Debugging – Part II

